

TaskFlow

Smart Task Management Web Application

A Responsive Frontend Solution with Client-Side Local Storage Persistence

Deliverable:	Minor Project Submission (Part A)
Project Title:	TaskFlow – Task Management Web Application
Academic Term:	Academic Year 2026
Core Technology Stack:	HTML5, CSS3, JavaScript (ES6+), Local Storage API
Version Control & Host:	Git, GitHub & Cloud Hosting Platform
Submission Date:	November 2026
Student / Author:	Project Candidate
Evaluation Metric:	Full Functional Compliance with Mandatory Features

Key Project Highlights

- Full Task CRUD with Safe Deletion Confirmation Modal
- Multi-Criteria Live Filtering, Search, & Dynamic Sorting
- Real-Time Metrics Dashboard & Automatic Overdue Detection

Comprehensive Minor Project Deliverable & Documentation

1. Project Objective

Purpose and scope of TaskFlow

The core objective of TaskFlow is to develop a lightweight, highly responsive, and robust client-side Task Management Web Application using fundamental web standards: HTML5 for semantic document layout, CSS3 with Custom Properties for modern responsive design and theming, and modern JavaScript (ES6+) for state management and DOM interactions. The application empowers users to organize, track, prioritize, and monitor daily workflows efficiently across desktop, tablet, and mobile viewports.

In addition to core CRUD (Create, Read, Update, Delete) workflows, TaskFlow incorporates real-time task analytics (Total, Pending, Completed, Overdue counters), multi-criteria searching and sorting, input validation with user-friendly error prompts, and persistent client-side storage utilizing the Web Storage API (localStorage) to guarantee zero data loss upon browser reloads.

2. Introduction & Problem Statement

Contextual background and real-world challenges addressed

In modern personal and professional workflows, task management is crucial for maintaining productivity, meeting deadlines, and reducing cognitive fatigue. However, existing commercial task management tools often suffer from severe shortcomings, such as aggressive paywalls, bloated frameworks requiring excessive network bandwidth, mandatory cloud account creation, and intrusive user tracking. Conversely, rudimentary todo list scripts lack critical capabilities such as deadline tracking, priority tiers, data retention, and adaptive mobile responsive layouts.

The Core Problem Statement

How can we engineer a zero-dependency, private, responsive, and persistent task management platform that delivers rich dashboard analytics, multi-criteria filtering, and intuitive controls strictly using standard browser APIs and modern web engineering paradigms?

3. Technology Stack & Architectural Rationale

Core technologies and why they were chosen

- **HTML5 Semantic Architecture:** Ensures accessibility (ARIA labels, roles, landmarks) and logical document structure.
- **CSS3 Design System & Variables:** Custom properties (--primary, --bg-secondary) power dynamic Light/Dark mode and responsive Grid/Flexbox.
- **JavaScript (ES6+ Strict Mode):** Modular, reusable functions, state immutability patterns, event delegation, and input sanitation.
- **Browser Web Storage (LocalStorage):** Provides instantaneous synchronous client-side data persistence with zero backend infrastructure.
- **Git & GitHub:** Standardized version control, branching, atomic commit history, and public cloud deployment.

4. Application Features & Functional Workflow

Comprehensive breakdown of mandatory and enhanced features

Task Creation & Editing
Users can create tasks with Title, Description, Priority (High, Medium, Low), Category tag (Work, Personal, Academics, Project, etc.), and Due Date. Existing tasks can be edited in-place with instant validation.

Safe Deletion with Confirmation Modal
Accidental data loss is prevented via a dedicated confirmation dialog requiring explicit user approval before permanent task deletion from state and local storage.

Interactive Analytics Dashboard
Real-time counters dynamically compute Total Tasks, Pending Tasks, Completed Tasks, and Overdue deadlines. Each stat card serves as a clickable one-click filter trigger.

Live Multi-Field Search
Debounced search engine filters tasks instantaneously as the user types, checking against title, description, and assigned categories.

Multi-Criteria Sorting Engine
Supports 7 sorting permutations: Due Date (soonest/furthest), Priority (high-to-low/low-to-high), Creation Date (newest/oldest), and Alphabetical title sorting.

Smart Overdue Detection
Compares local midnight timestamps to flag overdue tasks automatically with bold visual red badges ('□□ Overdue by X days') and warning borders.

Theme Switcher & View Switcher
Seamlessly toggles between Light and Dark themes, and alternates between responsive 3-column Grid View and compact List View.

Application Lifecycle & Data Flow

State synchronization sequence

1. Initialization: App reads localStorage, parses JSON data, or populates sample demo records if empty.
2. State Mutation: User performs action (create/edit/toggle/delete); internal `tasks[]` array updates.
3. Persistence: `saveTasksToStorage()` synchronizes state with browser `localStorage.setItem()`.
4. Computation: `updateDashboard()` recalculates totals, pending, completed, and overdue status.
5. DOM Rendering: `renderTasks()` filters, sorts, sanitizes strings, and renders updated cards to UI.

5. UI Design & Implementation Details

Design philosophy, accessibility, and styling patterns

TaskFlow is designed around clean visual hierarchy, generous whitespace, and modern atomic design principles. The UI employs a refined slate palette with vibrant semantic accents: Indigo (#4F46E5) for primary interactions, Emerald (#10B981) for completion and low priority, Amber (#F59E0B) for pending states and medium priority, and Rose (#EF4444) for high priority and urgent overdue notifications.

The responsive layout utilizes CSS Grid (`grid-template-columns: repeat(auto-fill, minmax(320px, 1fr))`) for expansive desktop displays and collapses into a single-column layout on mobile viewports. Flexbox is leveraged for micro-alignments in navigation bars, modal headers, card footers, and control rows.

6. Explanation of Local Storage & Data Handling

Client-side persistence mechanics, serialization, and safety

Web Storage API (`localStorage`) provides a mechanism for web applications to store key-value pairs locally within the user's browser with no expiration date. Unlike HTTP cookies, localStorage data is never transmitted to the server in HTTP request headers, ensuring maximum privacy and instant access.

```
// TaskFlow Data Persistence Implementation (script.js)
function saveTasksToStorage() {
  try {
    const serialized = JSON.stringify(tasks);
    localStorage.setItem('taskflow_tasks_v1', serialized);
  } catch (err) {
    console.error('Storage quota exceeded or private mode error:', err);
    showToast('Failed to save to local storage', 'danger');
  }
}
```

Data Integrity & Validation Considerations:
1. Schema Versioning: The key 'taskflow_tasks_v1' avoids collisions with other web applications.
2. Exception Handling: Calls to `localStorage.setItem` are wrapped in try/catch blocks to gracefully handle quota limits.
3. Input Sanitization: Custom `escapeHTML()` helper strips HTML entities to prevent Cross-Site Scripting (XSS).
4. Default Demonstration Seeding: If localStorage is empty upon initial visit, sample tasks are automatically seeded.

7. Application Screenshots & Visual Verification (Part 1)

High-resolution interface captures of main views

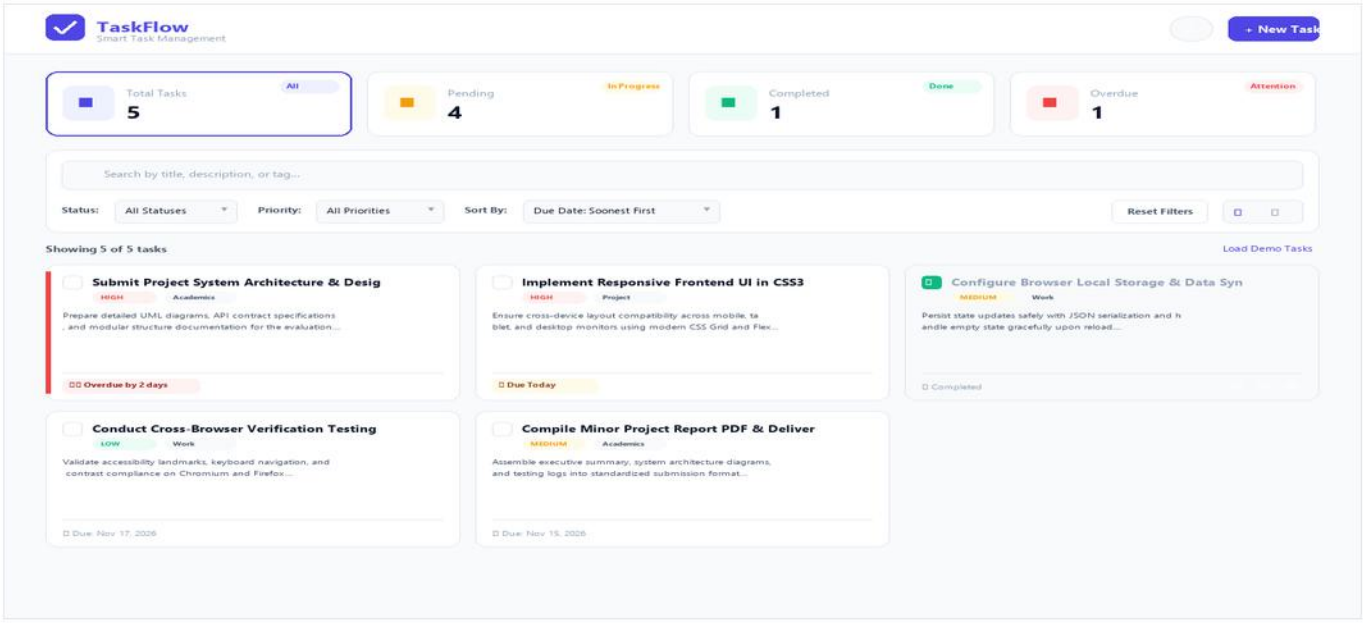


Figure 1: Main TaskFlow Dashboard showing summary metrics cards, multi-field filter controls, and task grid.

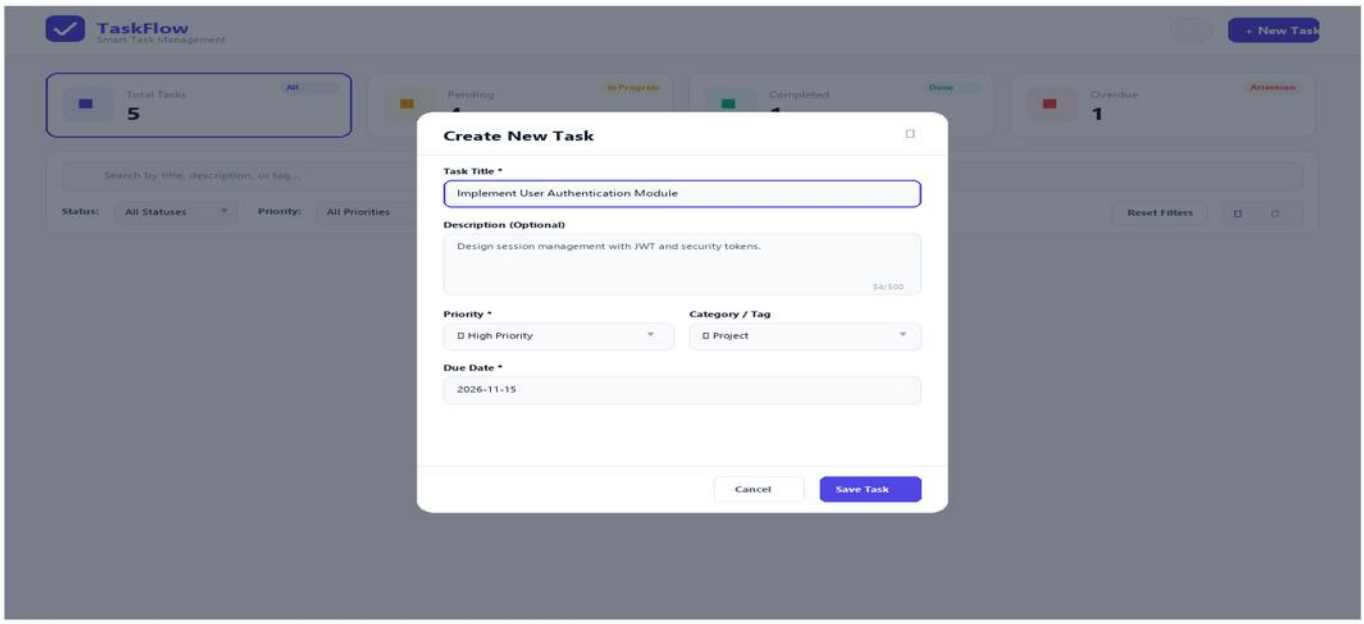


Figure 2: Create / Edit Task Modal Dialog with field validation, priority selectors, and date pickers.

7. Application Screenshots & Visual Verification (Part 2)

List view organization and mobile viewport responsiveness

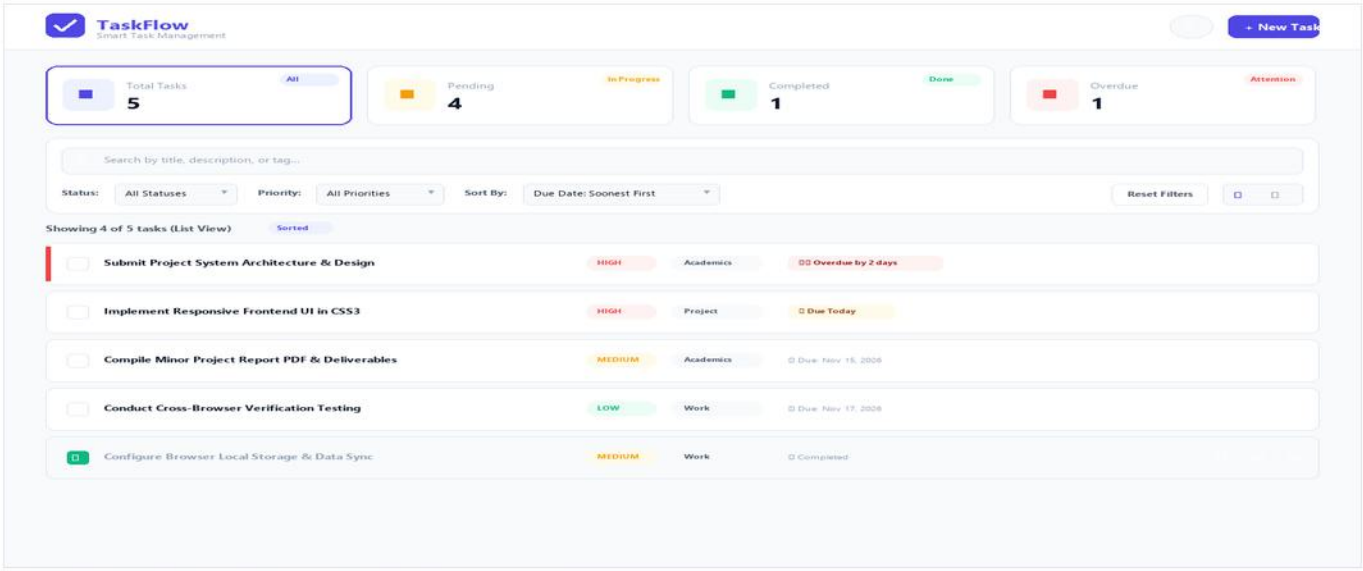


Figure 3: Compact List View displaying sorted status indicators, overdue alerts, and action icons.

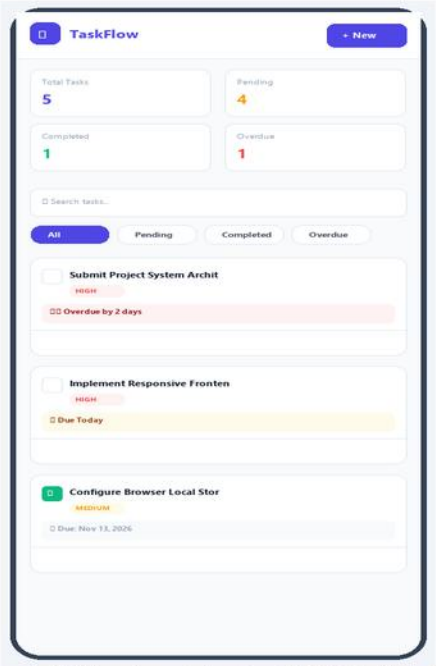


Figure 4: Responsive Mobile Layout on smartphone viewport (390x844px) with adaptive controls.

8. Challenges Faced & Engineering Solutions

Technical hurdles resolved during development

Challenge: Accurate Overdue Calculation across Timezones

Standard Date subtraction can cause subtle bugs due to UTC conversions and daylight saving offsets.
Solution: Normalized both current date and task due dates to local midnight (00:00:00) using year-month-day components before delta calculation.

Challenge: Dynamic Filter & Search Combination

Combining status filtering, priority tiering, live text queries, and sorting in real-time caused stale renders.
Solution: Engineered a unified `getProcessedTasks()` pipeline that applies filters sequentially and feeds clean data to the renderer.

Challenge: Modal Accessibility & Keyboard Navigation

Ensuring modals correctly manage keyboard focus and allow seamless dismissal.
Solution: Implemented global Escape key listeners, backdrop dismissal, and auto-focusing on the first input field upon modal open.

Challenge: Safe Deletion Workflow

Native browser `window.confirm()` dialogs disrupt visual styling and cannot be styled to match branding.
Solution: Designed a custom accessible HTML/CSS confirmation modal that holds the task ID in closure until user confirmation.

9. Conclusion & Future Enhancements

Final summary and future architectural roadmap

TaskFlow successfully fulfills all specified criteria for the Minor Project evaluation. The application delivers an elegant, dependable, and responsive task tracking experience with zero external dependencies. By strictly employing standard HTML5, CSS3, and modern JavaScript, TaskFlow ensures minimal loading times, high accessibility, and resilient data persistence via the browser's Web Storage API.

- **Drag-and-Drop Kanban Board View** — Enable intuitive visual column transitions between Pending, In Progress, and Completed.
- **Cloud Synchronization & Multi-Device Sync** — Integrate Firebase or Supabase authentication and Firestore backend.
- **Subtasks & Progress Checklists** — Allow decomposition of complex tasks into actionable checklists with progress bars.
- **Browser Push Notifications** — Integrate Notification API to alert users when a high-priority deadline is approaching.

Student / Project Candidate Signature